Code Bharat Backend Interview Guide - Akamai Prep Edition

Interview-focused revision version with speaking points, likely follow-up questions, and bug-fix reminders.

Goal: help you explain your backend confidently in interview, remember the flow quickly, and answer follow-up questions in simple English.

How To Use This For Akamai

Use it in 3 passes: first read only the interview boxes, then revise the request flow, then open the code sections for the files you feel weak in.

Best speaking formula: say responsibility -> where it sits in the request flow -> important logic -> one improvement. Even if you forget small code details, this keeps the answer strong.

Revise first	Why it matters
Application Entrypoint + Auth Middleware	Shows how requests enter the backend, how cookies are read, and how protected routes are enforced.
Auth Controllers + JWT Utility	Strong area for login flow, password hashing, token creation, and cookie settings.
Submission Controllers + Compiler Appendix	Best system-design story because it shows orchestration across DB, hidden test cases, and compiler service.

1. System Overview

This backend is a Node.js + Express + MongoDB API for a coding platform. It manages authentication, problems, hidden test cases, profile and leaderboard features, and solution judging through a separate compiler service.

2. High-Level Flow

```
Client
  -> Express app (Index.mjs)
    -> Global middleware (CORS, JSON parser, URL-encoded parser, cookie parser)
      -> Router
        -> Route middleware (validation, requireAuth, isAdmin)
          -> Controller
            -> Mongoose model / MongoDB
            -> External compiler service (submissions only)
          -> JSON response
```

3. Request Travel Example: Submit Solution

1. Frontend sends `POST /api/problem/:id/submit` with code and language.
2. Express runs CORS and parsers, then reads cookies.
3. `requireAuth` verifies the JWT cookie and loads the user.
4. `checkSchema(submissionValidatingSchema)` validates code and language.
5. `submitSolution` loads the problem and hidden test cases from MongoDB.
6. For each hidden test case, backend calls compiler service `/run`.
7. Backend compares normalized output with expected output.
8. Backend updates problem stats, user stats, and writes a submission record.
9. Backend returns the final verdict JSON.

4. Folder Structure

```
Backend/
 - .dockerignore
 - .env
 - .gitignore
 - Dockerfile
 - Index.mjs
 - package-lock.json
 - package.json
     - authControllers.mjs
     - problemControllers.mjs
     - profileControllers.mjs
     - submissionControllers.mjs
     - testCaseControllers.mjs
     - Database.mjs
     - isAdmin.mjs
     - requireAuth.mjs
     - Problem.mjs
     - Submission.mjs
     - Testcase.mjs
     - User.mjs
     - authRoutes.mjs
     - problemRoutes.mjs
     - profileRoutes.mjs
     - submissionRoutes.mjs
     - testCaseRoutes.mjs
         - loginValidatingSchema.mjs
         - registerValidatingSchema.mjs
       - problemValidatingSchema.mjs
       - profileValidatingSchema.mjs
       - submissionValidatingSchema.mjs
       - testCaseValidatingSchema.mjs
     - jwt.mjs
```

5. API Surface

Method	Path	Access	Chain
POST	/api/auth/register	Public	checkSchema(registerValidatingSchema) -> register
POST	/api/auth/login	Public	checkSchema(loginValidatingSchema) -> login
GET	/api/auth/me	Authenticated	requireAuth -> me
POST	/api/auth/logout	Cookie client	logout
GET	/api/problems	Public	getProblemSet
GET	/api/problem/:Id	Authenticated	requireAuth -> getProblem
POST	/api/problem	Admin	requireAuth -> isAdmin -> checkSchema(problemValidatingSchema) -> addProblem
DELETE	/api/problem/:Id	Admin	requireAuth -> isAdmin -> deleteProblem
POST	/api/testcases/:id	Admin	requireAuth -> isAdmin -> checkSchema(testCaseValidatingSchema) -> addTestCase
GET	/api/testcases/:id	Admin	requireAuth -> isAdmin -> getTestCasesByProblem

GET	/api/leaderboard	Public	getLeaderboard
GET	/api/user/:id	Public	userProfile
GET	/api/profile	Authenticated	requireAuth -> getMyProfile
PUT	/api/profile	Authenticated	requireAuth -> checkSchema(profileValidatingSchema) -> editProfile
GET	/api/submissions	Authenticated	requireAuth -> showAllSubmissions
GET	/api/problem/:id/submissions	Authenticated	requireAuth -> showProblemSubmissions
POST	/api/problem/:id/submit	Authenticated	requireAuth -> checkSchema(submissionValidatingSchema) -> submitSolution

6. Environment Variables

Variable	Use
PORT	Listen port
MONGODB_URL	MongoDB connection string
COOKIE_SECRET	Secret passed to cookie-parser
JWT_SECRET	JWT signing and verification secret
ADMIN_SECRET	Secret needed for admin registration
COMPILER_SERVICE_URL	Submission judge service URL
NODE_ENV	Controls secure cookie options
CORS_ALLOWED_ORIGINS	Optional comma-separated CORS allow-list override
SESSION_SECRET	Defined in .env but unused in source

7. Key Interview Risks / Observations

- No centralized error middleware.
- Submission stats and submission creation are not transactional.
- `jwt.mjs` has `dotenv.config;` instead of `dotenv.config();`.
- JWT payload uses `user.name`, but schema field is `userName`.
- Profile validator includes `firstName` and `lastName`, but model/controller do not persist them.
- Submission history endpoints do not use `try/catch`.
- There are no explicit performance indexes for heavy lookup paths like submissions and test cases.

Redacted `.env`

```
PORT=***REDACTED***
MONGODB_URL=***REDACTED***
COOKIE_SECRET=***REDACTED***
JWT_SECRET=***REDACTED***
SESSION_SECRET=***REDACTED***
ADMIN_SECRET=***REDACTED***
COMPILER_SERVICE_URL=***REDACTED***
```

Explain it simply in interview: This section explains the role of Redacted `.env` in the backend and where it sits in the overall request flow.

Likely Akamai follow-up questions:

- What responsibility does this file have?
- Where does it fit in the request flow?
- What improvement would you mention here?

One line to remember: Explain each file by responsibility, flow position, and one improvement.

Backend Package Manifest

File: Backend/package.json

Purpose: Defines Node version, runtime scripts, package mode, and core dependencies.

How it is used in request flow: This file shapes how the backend boots and what libraries are available across the app.

What to say in interview:

- Real startup uses `node Index.mjs` from the scripts section.
- `.mjs` files still run as ES modules even though package type is `commonjs`.
- Pinned `node: 20.x` helps deployment consistency.

Implementation notes:

- The `main` field is stale because the app really starts from `Index.mjs`.
- No lint or test scripts are defined yet.

Explain it simply in interview: This file does not handle API requests itself. It defines how the backend starts, which Node version it expects, and which libraries the whole app depends on.

Likely Akamai follow-up questions:

- Why is `Index.mjs` the real entry point even though `main` points somewhere else?
- Why pin a Node version for deployments?
- What problems happen when dependency versions differ across systems?

One line to remember: Package.json is the setup contract of the backend, not the business logic.

Code

```
 1: {
 2:   "name": "backend",
 3:   "version": "1.0.0",
 4:   "description": "",
 5:   "main": "index.js",
 6:   "engines": {
 7:     "node": "20.x"
 8:   },
 9:   "scripts": {
10:     "dev": "node Index.mjs",
11:     "start": "node Index.mjs",
12:     "test": "echo \"Error: no test specified\" && exit 1"
13:   },
14:   "keywords": [],
15:   "author": "",
16:   "license": "ISC",
17:   "type": "commonjs",
18:   "dependencies": {
19:     "axios": "^1.11.0",
20:     "bcryptjs": "^3.0.2",
21:     "cookie-parser": "^1.4.7",
22:     "cors": "^2.8.5",
23:     "dotenv": "^17.2.1",
```

```
24:     "express": "^5.1.0",
25:     "express-validator": "^7.2.1",
26:     "jsonwebtoken": "^9.0.2",
27:     "mongoose": "^8.17.0"
28:   }
29: }
```

Walkthrough

Lines 2-5: Package metadata starts here.

Lines 6-8: Pins the Node.js runtime version.

Lines 9-16: Defines start and dev commands.

Line 17: Sets default package mode, although `.mjs` files override it.

Lines 18-29: Declares the backend stack: Express, Mongoose, bcrypt, JWT, cookies, validation, axios, CORS, dotenv.

Git Ignore Rules

File: Backend/.gitignore

Purpose: Prevents secrets, dependencies, logs, and local editor files from being committed.

How it is used in request flow: Not part of request execution, but critical for secure repository hygiene.

What to say in interview:

- Ignoring `.env` prevents secret leakage.
- Ignoring `node_modules` and local editor folders keeps the repo clean.

Implementation notes:

- `package-lock.json` is ignored here even though a lockfile exists in the repo.

Explain it simply in interview: This file improves safety and cleanliness. It prevents secrets, heavy folders, and local machine files from entering Git history or Docker build context.

Likely Akamai follow-up questions:

- Why should `.env` never be committed?
- What is the difference between `.gitignore` and `.dockerignore`?
- How do these files improve security and build speed?

One line to remember: Ignore rules do not change runtime behavior, but they protect the repo and deployment pipeline.

Code

```
 1: node_modules
 2: .env
 3: logs/
 4: *.log
 5: npm-debug.log*
 6: yarn-debug.log*
 7: yarn-error.log*
 8: pnpm-debug.log*
 9:
10: .DS_Store
11: Thumbs.db
12:
13: package-lock.json
14: yarn.lock
15: pnpm-lock.yaml
16:
17: coverage/
18: dist/
19: build/
20:
21: pids/
22: *.pid
23: *.seed
24: *.pid.lock
25: dump.rdb
26:
27: .vscode/
28: .idea/
29: *.swp
30:
```

Walkthrough

Line 1: Skips installed dependencies.

Line 2: Skips secret environment files.

Lines 3-16: Skips runtime log files.

Lines 17-26: Skips generated coverage and build output.

Lines 27-30: Skips editor-specific files.

Docker Ignore Rules

File: Backend/.dockerignore

Purpose: Keeps the Docker build context small and secret-safe.

How it is used in request flow: Affects container build behavior rather than live request handling.

What to say in interview:

- Skipping `node_modules` avoids copying host-specific packages into the image.
- Skipping `.env` keeps secrets out of Docker layers.

Implementation notes:

- Very small but high-value Docker ignore file.

Explain it simply in interview: This file improves safety and cleanliness. It prevents secrets, heavy folders, and local machine files from entering Git history or Docker build context.

Likely Akamai follow-up questions:

- Why should `.env` never be committed?
- What is the difference between `.gitignore` and `.dockerignore`?
- How do these files improve security and build speed?

One line to remember: Ignore rules do not change runtime behavior, but they protect the repo and deployment pipeline.

Code

```
1: node_modules
2: .env
```

Walkthrough

Line 1: Prevents local modules from entering the image context.

Line 2: Prevents environment secrets from entering the image context.

Backend Dockerfile

File: Backend/Dockerfile

Purpose: Builds a runnable container image for the backend.

How it is used in request flow: Defines the environment in which the Express server will run in containerized deployments.

What to say in interview:

- Uses Node 20, matching `package.json` engines.
- Copies package files before source to improve Docker layer caching.
- Starts the app with `node Index.mjs`.

Implementation notes:

- Could be improved with `npm ci`, a non-root user, and a slimmer production image.

Explain it simply in interview: This file turns the backend into a portable container image. It defines the runtime image, dependency installation steps, copied files, exposed port, and startup command.

Likely Akamai follow-up questions:

- Why copy `package*.json` before copying the whole source?
- Why is `npm ci` often preferred in production builds?
- What hardening would you add for a safer production container?

One line to remember: The Dockerfile makes the same backend run predictably on any machine.

Code

```
 1: FROM node:20
 2:
 3: WORKDIR /app
 4:
 5: COPY package*.json ./
 6:
 7: RUN npm install
 8:
 9: COPY . .
10:
11: EXPOSE 8000
12:
13: ENV NODE_ENV=production
14:
15: CMD ["node", "Index.mjs"]
```

Walkthrough

Lines 1-2: Selects the Node 20 base image.

Lines 3-4: Sets the working directory inside the container.

Lines 5-6: Copies package metadata first for cached installs.

Lines 7-8: Installs backend dependencies inside the image.

Lines 9-10: Copies application source code.

Lines 11-12: Documents the app port.

Lines 13-14: Marks the runtime as production.

Line 15: Starts the backend server.

Application Entrypoint

File: Backend/Index.mjs

Purpose: Bootstraps the server, database connection, CORS, parsers, routers, health route, 404 fallback, and listener.

How it is used in request flow: Every request first enters here, passes global middleware, then reaches a router/controller or the 404 handler.

What to say in interview:

- Global middleware order is important: CORS, body parsing, cookie parsing, then routers.
- The app uses cookie-based auth, so CORS is configured with credentials.
- Routers include full API paths and are mounted without a shared prefix.

Implementation notes:

- There is no central error middleware.
- Cookie parser receives a secret, but the auth cookie is not stored as a signed cookie.

Explain it simply in interview: This is the first backend file that runs. It creates the Express app, connects MongoDB, sets middleware, mounts routes, and starts the server.

Likely Akamai follow-up questions:

- Why does middleware order matter in Express?
- Why is `credentials: true` needed when auth uses cookies?
- Where would you add global error handling and logging?

One line to remember: Every request enters through `Index.mjs`, so this file defines the app pipeline.

Code

```
 1: import express from 'express';
 2: import dotenv from 'dotenv';
 3: import cookieParser from 'cookie-parser';
 4: import DBConnection from './src/Database/Database.mjs';
 5: import authRouter from './src/Routes/authRoutes.mjs';
 6: import problemRouter from './src/Routes/problemRoutes.mjs';
 7: import testCaseRouter from './src/Routes/testCaseRoutes.mjs';
 8: import profileRouter from './src/Routes/profileRoutes.mjs';
 9: import submissionRouter from './src/Routes/submissionRoutes.mjs';
10: import cors from "cors";
11:
12: dotenv.config();
13: const app = express();
14:
15: DBConnection();
16:
17: const defaultAllowedOrigins = [
18:   "http://localhost:5173",
19:   "http://127.0.0.1:5173",
20:   "https://code-bharat-one.vercel.app",
21:   "https://jatsamajdirectory.com",
22: ];
23:
24: const allowedOrigins = process.env.CORS_ALLOWED_ORIGINS
25:   ? process.env.CORS_ALLOWED_ORIGINS.split(",").map((origin) => origin.trim()).filter(Boolean)
26:   : defaultAllowedOrigins;
27:
28: const isAllowedOrigin = (origin) => {
29:   if (allowedOrigins.includes(origin)) {
30:     return true;
31:   }
32:
33:   try {
34:     return new URL(origin).hostname.endsWith(".onrender.com");
35:   } catch {
```

```
36:     return false;
37:   }
38: };
39:
40: app.use(cors({
41:   origin: (origin, callback) => {
42:     if (!origin || isAllowedOrigin(origin)) {
43:       return callback(null, true);
44:     }
45:
46:     return callback(new Error("Origin not allowed by CORS"));
47:   },
48:   credentials: true
49: }));
50:
51: app.use(express.json());
52: app.use(express.urlencoded({ extended: true }));
53: app.use(cookieParser(process.env.COOKIE_SECRET));
54:
55: app.use(authRouter);
56: app.use(problemRouter);
57: app.use(testCaseRouter);
58: app.use(profileRouter);
59: app.use(submissionRouter);
60:
61: app.get("/", (req, res) => {
62:   res.json({ message: "Code Bharat Backend API is live" });
63: });
64:
65: app.use((req, res) => {
66:   res.status(404).json({ message: 'Route not found' });
67: });
68:
69: const PORT = process.env.PORT || 8000;
70: app.listen(PORT, () => {
71:   console.log(`Server running on http://localhost:${PORT}`);
72: });
```

Walkthrough

Lines 1-11: Imports the server framework and all route/config modules needed to boot the app.

Line 12: Loads environment variables from `.env`.

Lines 13-14: Creates the Express application instance.

Lines 15-16: Starts MongoDB connection at boot time.

Lines 17-23: Defines the hardcoded frontend allow list.

Lines 24-27: Allows deployments to override the CORS list with an env var.

Lines 28-39: Adds custom logic for allowed origins, including Render preview domains.

Lines 40-50: Registers credential-aware CORS before any route logic.

Lines 51-54: Registers JSON parser, URL-encoded parser, and cookie parser.

Lines 55-60: Mounts each feature router.

Lines 61-64: Defines a basic health-check endpoint.

Lines 65-68: Defines a catch-all 404 handler.

Lines 69-72: Chooses the port and starts the listener.

Database Connection Module

File: Backend/src/Database/Database.mjs

Purpose: Connects Mongoose to MongoDB.

How it is used in request flow: Runs during startup before routes begin handling database-backed requests.

What to say in interview:

- Database connection is wrapped in a dedicated function for cleaner bootstrapping.
- Mongoose is the abstraction layer over MongoDB.

Implementation notes:

- Connection errors are logged but not re-thrown, so the server can continue running even if MongoDB is unavailable.

Explain it simply in interview: This file centralizes MongoDB connection logic so the app has one clean place to open the database connection during startup.

Likely Akamai follow-up questions:

- Why keep DB connection logic in one module?
- What should happen if the database is unavailable at startup?
- Would you retry the connection or fail fast in production?

One line to remember: One DB connector keeps startup logic simple and avoids repeated connection code.

Code

```
 1: import mongoose from 'mongoose';
 2: import dotenv from 'dotenv';
 3:
 4: dotenv.config();
 5:
 6: const DBConnection = async () => {
 7:
 8:     const MONGODB_URL = process.env.MONGODB_URL;
 9:
10:     try {
11:         await mongoose.connect(MONGODB_URL);
12:         console.log("Connected to the Database Successfully,")
13:     } catch (error) {
14:         console.log("Error connecting to database " , error.message);
15:     }
16: };
17:
18: export default DBConnection;
19:
```

Walkthrough

Lines 1-3: Imports Mongoose and dotenv.

Lines 4-5: Loads environment variables for the database connection.

Lines 6-7: Defines the async connection function.

Lines 8-10: Reads the MongoDB connection string.

Line 11: Opens the database connection.

Lines 12-13: Logs successful connection.

Lines 14-17: Logs connection failure.

Lines 18-19: Exports the function for `Index.mjs`.

JWT Utility

File: Backend/src/Utils/jwt.mjs

Purpose: Centralizes token creation and verification for cookie-based authentication.

How it is used in request flow: Used by login to issue tokens and by auth middleware to verify them on protected requests.

What to say in interview:

- Token payload contains identity and role data.
- Token expiry is aligned with cookie lifetime at two hours.

Implementation notes:

- Bug: `dotenv.config;` is missing parentheses.
- Bug: payload uses `user.name`, but the schema field is `userName`.

Explain it simply in interview: This file creates and verifies JWT tokens for cookie-based authentication. Login issues the token, and protected requests verify it before continuing.

Likely Akamai follow-up questions:

- What data do you keep in the JWT payload and why?
- Why use HTTP-only cookies instead of localStorage for auth?
- What real bug exists in this file and what could it break?

One line to remember: JWT proves identity, while the cookie is how that token travels on each request.

Code

```
 1: import jwt from "jsonwebtoken"
 2: import dotenv from "dotenv";
 3:
 4: dotenv.config;
 5:
 6: const JWT_SECRET = process.env.JWT_SECRET;
 7:
 8: export const createToken = (user) => {
 9:     return jwt.sign(
10:         {
11:             id: user._id,
12:             name: user.name,
13:             email: user.email,
14:             role: user.role
15:         },
16:         JWT_SECRET,
17:         { expiresIn: '2h' }
18:     );
19: };
20:
21: export const verifyToken = (token) => {
22:     return jwt.verify(token , JWT_SECRET);
23: };
```

Walkthrough

Lines 1-3: Imports JWT signing support and dotenv.

Lines 4-5: Attempts to load env vars, but this line does not actually call the function.

Lines 6-7: Reads the JWT signing secret.

Lines 8-16: Builds a signed JWT from user details.

Lines 17-20: Sets token expiration.

Lines 21-23: Verifies a token and returns decoded payload.

Authentication Middleware

File: Backend/src/Middleware/requireAuth.mjs

Purpose: Protects routes by reading the `token` cookie, verifying it, loading the user, and attaching `req.user`.

How it is used in request flow: Runs before protected controllers such as `me`, `getProblem`, `getMyProfile`, and submission endpoints.

What to say in interview:

- Middleware validates the cookie token and also re-reads the user from MongoDB.
- Controllers reuse `req.user` instead of re-querying identity again.

Implementation notes:

- This depends on frontend requests being sent with cookies and `credentials: true`.

Explain it simply in interview: This middleware protects private routes. It reads the auth cookie, verifies the token, fetches the user, and attaches the user object to `req` for later handlers.

Likely Akamai follow-up questions:

- Why attach `req.user` instead of querying the database again in every controller?
- What response should be returned if the cookie is missing or invalid?
- How is authentication different from authorization?

One line to remember: Middleware is a gatekeeper that blocks bad requests before controller logic runs.

Code

```
 1: import { verifyToken } from "../Utils/jwt.mjs";
 2: import { User } from '../Models/User.mjs';
 3:
 4: const requireAuth = async (req, res, next) => {
 5:   const token = req.cookies.token;
 6:
 7:   if (!token) {
 8:     return res.status(401).json({ message: "No token provided" });
 9:   }
10:
11:   try {
12:     const decoded = verifyToken(token);
13:
14:     const user = await User.findById(decoded.id).select('-password');
15:     if (!user) {
16:       return res.status(404).json({ message: "User not found" });
17:     }
18:
19:     req.user = user;
20:     next();
21:   } catch (err) {
22:     return res.status(401).json({ message: "Invalid or expired token" });
23:   }
24: };
25:
26: export default requireAuth;
```

Walkthrough

Lines 1-3: Imports token verification and the User model.

Line 4: Defines the route-protection middleware.

Lines 5-6: Reads the auth token from cookies.

Lines 7-11: Rejects requests when no token is present.

Lines 12-13: Decodes and verifies the JWT.

Lines 14-18: Loads the user and hides the password field.

Lines 19-21: Attaches the authenticated user to the request.

Lines 22-26: Handles invalid or expired tokens.

Admin Authorization Middleware

File: Backend/src/Middleware/isAdmin.mjs

Purpose: Allows only admin users to proceed to sensitive routes.

How it is used in request flow: Runs after `requireAuth` on admin-only routes like problem creation and test-case management.

What to say in interview:

- Authentication proves identity; authorization checks permission.
- Role comparison is lowercase, matching the model's pre-save normalization.

Implementation notes:

- If middleware order changes and `req.user` is missing, this denies access.

Explain it simply in interview: This middleware checks whether an already authenticated user has admin access before sensitive actions like create or delete are allowed.

Likely Akamai follow-up questions:

- Why separate normal auth from admin auth?
- Which endpoints in this project should be admin-only?
- Is it safer to trust the role in JWT alone or re-check the DB?

One line to remember: Authentication says who the user is; authorization says what the user can do.

Code

```
1: const isAdmin = (req, res, next) => {
2:   if (req.user && req.user.role.toLowerCase() === 'admin') {
3:     return next();
4:   }
5:   return res.status(403).json({ message: "Admin access only" });
6: };
7:
8: export default isAdmin;
```

Walkthrough

Line 1: Defines the admin guard.

Lines 2-4: Allows the request only for authenticated admins.

Lines 5-7: Rejects non-admin users.

Line 8: Exports the middleware.

Auth Controllers

File: Backend/src/Controllers/authControllers.mjs

Purpose: Implements register, login, logout, and current-user responses.

How it is used in request flow: These functions run after route validation and, for `me`, after auth middleware has already attached `req.user`.

What to say in interview:

- Register hashes passwords before storage.
- Login reuses `req._foundUser` from validator to avoid a duplicate lookup.
- Auth state is stored in an HTTP-only cookie.

Implementation notes:

- Login returns 201 instead of 200.
- Cookie settings change by `NODE_ENV` to support cross-site production cookies.

Explain it simply in interview: This controller file contains the main business logic for auth. Routes send validated requests here, and controllers talk to models, utilities, and the response object.

Likely Akamai follow-up questions:

- What belongs in a controller and what should stay in middleware?
- Why validate input before controller logic starts?
- What improvement would you mention first in this controller?

One line to remember: Controllers are where request data turns into actual application behavior.

Code

```
import { matchedData, validationResult } from "express-validator";
import bcrypt from "bcryptjs";
import { User } from "../Models/User.mjs";
import { createToken } from "../Utils/jwt.mjs";

const getCookieOptions = () => {
    const isProduction = process.env.NODE_ENV === "production";

    return {
        httpOnly: true,
        maxAge: 1000 * 60 * 60 * 2,
        secure: isProduction,
        sameSite: isProduction ? "None" : "Lax",
        path: "/",
    };
};

const getClearCookieOptions = () => {
    const isProduction = process.env.NODE_ENV === "production";

    return {
        httpOnly: true,
        secure: isProduction,
        sameSite: isProduction ? "None" : "Lax",
        path: "/",
    };
};

export const register = async (req, res) => {
    const errors = validationResult(req);

    if (!errors.isEmpty()) {
        return res.status(400).json({ errors: errors.array() });
    }

    try {
        const data = matchedData(req, { includeOptionals: true });
```

```
38:         let { password, role, adminSecret, ...restData } = data;
39:
40:         role = role ? role.toLowerCase() : "user";
41:
42:         const hashedPassword = await bcrypt.hash(password, 10);
43:
44:         const user = await User.create({
45:             ...restData,
46:             role,
47:             password: hashedPassword
48:         });
49:
50:         const { password: _password, ...publicUser } = user.toObject();
51:
52:         res.status(201).json({ message: 'Registered successfully', user: publicUser });
53:     } catch (error) {
54:         return res.status(500).json({ message: 'Internal server error', error });
55:     }
56: };
57:
58: export const login = async (req, res) => {
59:     const errors = validationResult(req);
60:
61:     if (!errors.isEmpty()) {
62:         return res.status(400).json({ errors: errors.array() });
63:     }
64:
65:     try {
66:         const user = req._foundUser;
67:         const token = createToken(user);
68:
69:         res.cookie('token', token, getCookieOptions());
70:
71:         res.status(201).json({ message: 'Login successful' });
72:     } catch (error) {
73:         return res.status(500).json({ message: 'Internal server error', error });
74:     }
75: };
76:
77: export const me = async (req, res) => {
78:     const { password, ...publicUser } = req.user.toObject();
79:     res.status(200).json({ user: publicUser });
80: };
81:
82: export const logout = async (req, res) => {
83:     res.clearCookie('token', getClearCookieOptions());
84:     res.status(200).json({ message: "Logged out successfully" });
85: };
```

Walkthrough

Lines 6-17: Builds cookie settings for login.

Lines 18-28: Builds matching settings for logout cookie clearing.

Line 29: Starts registration flow.

Lines 30-36: Stops if route validation failed.

Lines 37-41: Uses only validated request fields.

Lines 42-43: Hashes the password before storing it.

Lines 44-49: Creates the new user document.

Lines 50-57: Removes password hash from API response.

Lines 58-65: Starts login flow.

Lines 66-68: Reuses user loaded by login validation.

Lines 69-76: Writes JWT into an HTTP-only cookie.

Lines 77-81: Returns authenticated user profile data.

Lines 82-85: Clears the auth cookie.

Problem Controllers

File: Backend/src/Controllers/problemControllers.mjs

Purpose: Handles reading, creating, listing, and deleting problem documents.

How it is used in request flow: Controllers receive validated/authenticated requests from problem routes and then perform Mongoose operations.

What to say in interview:

- ObjectId validation happens before database access.
- Create route records `created_by` from the authenticated admin.
- Delete route manually cleans related test cases and submissions.

Implementation notes:

- Problem list uses field projection for lighter responses.

Explain it simply in interview: This controller file contains the main business logic for problem. Routes send validated requests here, and controllers talk to models, utilities, and the response object.

Likely Akamai follow-up questions:

- What belongs in a controller and what should stay in middleware?
- Why validate input before controller logic starts?
- What improvement would you mention first in this controller?

One line to remember: Controllers are where request data turns into actual application behavior.

Code

```
 1: import { matchedData, validationResult } from "express-validator";
 2: import { Problem } from "../Models/Problem.mjs";
 3: import mongoose from "mongoose";
 4: import { TestCase } from "../Models/Testcase.mjs";
 5: import { Submission } from "../Models/Submission.mjs";
 6:
 7:
 8: export const getProblem = async (req, res) => {
 9:     const { Id } = req.params;
10:
11:     if (!mongoose.Types.ObjectId.isValid(Id)) {
12:         return res.status(400).json({ message: 'Invalid MongoDB ID' });
13:     }
14:
15:     try {
16:         const problem = await Problem.findById(Id);
17:
18:         if (!problem) {
19:             return res.status(404).json({ message: 'Problem not found' });
20:         }
21:
22:         res.status(200).json(problem);
23:     } catch (error) {
24:         return res.status(500).json({ message: 'Failed to fetch problem', error: error.message });
25:     }
26: };
27:
28: export const addProblem = async (req, res) => {
29:     const errors = validationResult(req);
30:
31:     if (!errors.isEmpty()) {
32:         return res.status(400).json({ errors: errors.array() });
33:     }
34:
35:     try {
36:         const data = matchedData(req, { includeOptionals: true });
37:
38:         const created_by = req.user.id;
```

```
39:
40:         const problem = await Problem.create({
41:             ...data,
42:             created_by
43:         });
44:
45:         res.status(201).json({ message: 'Problem added successfully', problem });
46:     } catch (error) {
47:         console.error("Add Problem Error:", error.stack || error.message || error);
48:         return res.status(500).json({ message: 'Internal server error', error: error.message });
49:     }
50: };
51:
52: export const getProblemSet = async (req, res) => {
53:     try {
54:         const problems = await Problem.find().select('problemNumber title acceptanceRate');
55:
56:         res.status(200).json({ problems });
57:     } catch (error) {
58:         return res.status(500).json({ message: 'Failed to fetch problems', error });
59:     }
60: };
61:
62: export const deleteProblem = async (req, res) => {
63:     const Id = req.params.Id;
64:
65:     if (!mongoose.Types.ObjectId.isValid(Id)) {
66:         return res.status(400).json({ message: "Invalid problem ID" });
67:     }
68:
69:     try {
70:         const deletedProblem = await Problem.findByIdAndDelete(Id);
71:
72:         if (!deletedProblem) {
73:             return res.status(404).json({ message: "Problem not found" });
74:         }
75:
76:         await TestCase.deleteMany({ problemId: Id });
77:         await Submission.deleteMany({ problemId: Id });
78:
79:         res.status(200).json({ message: "Problem and related data deleted successfully" });
80:     } catch (err) {
81:         console.error("Delete Problem Error:", err);
82:         res.status(500).json({ message: "Internal server error" });
83:     }
84: };
```

Walkthrough

Lines 8-10: Starts single-problem read logic.

Lines 11-15: Rejects malformed ids early.

Lines 16-27: Loads one problem document.

Lines 28-35: Starts create-problem flow.

Lines 36-37: Extracts validated body fields.

Lines 38-39: Uses authenticated admin id as creator.

Lines 40-51: Creates the problem.

Lines 52-53: Starts lightweight list flow.

Lines 54-61: Fetches only list-page fields.

Lines 62-69: Starts delete flow.

Lines 70-75: Deletes main problem document.

Line 76: Deletes related hidden test cases.

Lines 77-84: Deletes related submission history.

Profile Controllers

File: Backend/src/Controllers/profileControllers.mjs

Purpose: Handles public profile reads, self-profile reads, leaderboard generation, and profile editing.

How it is used in request flow: Reads user documents directly or, for self-profile routes, uses authenticated identity from middleware.

What to say in interview:

- Leaderboard is computed in application code after fetching user data with `.lean()`.
- Profile edit updates only selected fields.

Implementation notes:

- Validator includes `firstName` and `lastName`, but controller/model do not use them.

Explain it simply in interview: This controller file contains the main business logic for profile. Routes send validated requests here, and controllers talk to models, utilities, and the response object.

Likely Akamai follow-up questions:

- What belongs in a controller and what should stay in middleware?
- Why validate input before controller logic starts?
- What improvement would you mention first in this controller?

One line to remember: Controllers are where request data turns into actual application behavior.

Code

```
  1: import { validationResult } from "express-validator";
  2: import { User } from "../Models/User.mjs";
  3:
  4: export const userProfile = async (req, res) => {
  5:   const { id } = req.params;
  6:
  7:   try {
  8:     const user = await User.findById(id).select(
  9:       'userName profilePicture bio location institution problemsSolved totalSubmissions
acceptedSubmissions'
 10:     );
 11:
 12:     if (!user) {
 13:       return res.status(404).json({ message: "User not found" });
 14:     }
 15:
 16:     res.status(200).json({ user });
 17:   } catch (error) {
 18:     console.error("Error fetching user profile:", error.message);
 19:     res.status(500).json({ message: "Internal server error" });
 20:   }
 21: };
 22:
 23: export const getMyProfile = async (req, res) => {
 24:   try {
 25:     const user = await User.findById(req.user.id).select(
 26:       'userName email profilePicture bio location institution problemsSolved totalSubmissions
acceptedSubmissions'
 27:     );
 28:
 29:     if (!user) {
 30:       return res.status(404).json({ message: "User not found" });
 31:     }
 32:
 33:     res.status(200).json({ user });
 34:   } catch (error) {
 35:     console.error("Error in getMyProfile:", error.message);
 36:     res.status(500).json({ message: "Internal server error" });
 37:   }
 38: };
```

```

export const getLeaderboard = async (_req, res) => {
  try {
    const users = await User.find()
      .select(
        "userName profilePicture institution location problemsSolved totalSubmissions acceptedSubmissions"
      )
      .lean();

    const leaderboard = users
      .map((user) => {
        const solved = Array.isArray(user.problemsSolved) ? user.problemsSolved.length : 0;
        const totalSubmissions = user.totalSubmissions ?? 0;
        const acceptedSubmissions = user.acceptedSubmissions ?? 0;
        const acceptanceRate = totalSubmissions
          ? Number(((acceptedSubmissions / totalSubmissions) * 100).toFixed(1))
          : 0;

        return {
          rank: 0,
          _id: user._id,
          userName: user.userName,
          profilePicture: user.profilePicture,
          institution: user.institution,
          location: user.location,
          solved,
          totalSubmissions,
          acceptedSubmissions,
          acceptanceRate,
        };
      })
      .sort((a, b) => {
        if (b.solved !== a.solved) return b.solved - a.solved;
        if (b.acceptedSubmissions !== a.acceptedSubmissions) {
          return b.acceptedSubmissions - a.acceptedSubmissions;
        }
        if (a.totalSubmissions !== b.totalSubmissions) {
          return a.totalSubmissions - b.totalSubmissions;
        }
        return a.userName.localeCompare(b.userName);
      })
      .map((user, index) => ({
        ...user,
        rank: index + 1,
      }));

    res.status(200).json({ leaderboard });
  } catch (error) {
    console.error("Error fetching leaderboard:", error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};

export const editProfile = async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }

  const userId = req.user.id;
  const { profilePicture, bio, location, institution } = req.body;

  try {
    const user = await User.findById(userId);

    if (!user) {
      return res.status(404).json({ message: "User not found" });
    }

    if (profilePicture !== undefined) user.profilePicture = profilePicture;
    if (bio !== undefined) user.bio = bio;
    if (location !== undefined) user.location = location;
    if (institution !== undefined) user.institution = institution;

```

```
113:     await user.save();
114:
115:     res.status(200).json({ message: "Profile updated successfully", user });
116:   } catch (error) {
117:     console.error("Edit Profile Error:", error.message);
118:     res.status(500).json({ message: "Internal server error" });
119:   }
120: };
```

Walkthrough

Lines 4-7: Starts public profile lookup.

Lines 8-22: Fetches selected public profile fields.

Lines 23-24: Starts authenticated self-profile lookup.

Lines 25-39: Fetches selected current-user fields.

Lines 40-41: Starts leaderboard generation.

Lines 42-47: Loads users needed for ranking.

Lines 48-91: Builds, sorts, and ranks the leaderboard.

Lines 92-98: Starts profile update flow.

Lines 99-112: Reads editable fields from request body.

Lines 113-120: Persists profile changes.

Submission Controllers

File: Backend/src/Controllers/submissionControllers.mjs

Purpose: Implements submission history and the full online-judge flow.

How it is used in request flow: This is the deepest request path: authenticated request -> problem lookup -> test-case lookup -> compiler calls -> comparison -> stats updates -> submission write -> response.

What to say in interview:

- Good interview example of orchestration across middleware, models, and an external service.
- Whitespace normalization reduces false-negative comparisons.
- Writes problem stats, user stats, and submission history in one request.

Implementation notes:

- History endpoints do not use `try/catch`.
- Stats updates are not transactional.
- Storing all judge inputs/outputs in submissions helps debugging but increases storage.

Explain it simply in interview: This is the main orchestration file of the whole project. It handles the full online judge flow: validate input, load problem and hidden test cases, call the compiler service, compare outputs, update stats, and store the submission.

Likely Akamai follow-up questions:

- Walk me through one submission from request to final verdict.
- What happens if the compiler service fails halfway through?
- Why are transactions, retries, or queues useful here?
- How would you optimize this path if traffic grows a lot?

One line to remember: This file is the orchestration brain of the platform and your strongest interview story.

Code

```
import { matchedData, validationResult } from "express-validator";
import mongoose from "mongoose";
import axios from "axios";
import { Submission } from "../Models/Submission.mjs";
import { Problem } from "../Models/Problem.mjs";
import { TestCase } from "../Models/Testcase.mjs";
import { User } from "../Models/User.mjs";

const normalize = (s) => s.trim().split("\n").map((l) => l.trim()).join("\n");

export const showAllSubmissions = async (req, res) => {
  if (!req.user) return res.status(401).json({ message: "Unauthorized" });

  const submissions = await Submission.find({ userId: req.user._id })
    .select("problemId problemNumber language status createdAt")
    .sort({ createdAt: -1 })
    .lean();

  res.json({ submissions });
};

export const showProblemSubmissions = async (req, res) => {
  const { id } = req.params;
  if (!mongoose.Types.ObjectId.isValid(id))
    return res.status(400).json({ message: "Invalid MongoDB ID" });

  if (!req.user) return res.status(401).json({ message: "Unauthorized" });

  const submissions = await Submission.find({
    userId: req.user._id,
    problemId: id,
  })
```

```
    .select("problemId problemNumber language status createdAt")
    .sort({ createdAt: -1 })
    .lean();

  res.json({ submissions });
};

export const submitSolution = async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty())
    return res.status(400).json({ message: "Validation failed", errors: errors.array() });

  const problemId = req.params.id;
  if (!mongoose.Types.ObjectId.isValid(problemId))
    return res.status(400).json({ message: "Invalid problem ID" });

  const { code, language } = matchedData(req);

  try {
    console.log("[1] Finding problem:", problemId);
    const problem = await Problem.findById(problemId);
    if (!problem) return res.status(404).json({ message: "Problem not found" });

    console.log("[2] Fetching test cases...");
    const testCases = await TestCase.find({ problemId });
    if (!testCases.length)
      return res.status(400).json({ message: "No test cases found" });

    let passed = 0;
    const compilerUrl = process.env.COMPILER_SERVICE_URL || "http://localhost:7000";

    console.log("[3] Compiler URL:", compilerUrl);
    for (const tc of testCases) {
      console.log("[4] Sending test case input:", tc.input);
      try {
        const runRes = await axios.post(
          `${compilerUrl}/run`,
          { code, language, input: tc.input },
          { timeout: 10000 }
        );
        console.log("[5] Compiler response:", runRes.data);

        const actual = normalize(runRes.data.output);
        const expected = normalize(tc.output);
        if (actual === expected) passed++;
      } catch (err) {
        console.error("[6] Test case failed:", tc.input);
        console.error("[6] Error message:", err.message);
        console.error("[6] Response:", err?.response?.data);
        return res.status(500).json({
          message: "Error while running test cases",
          error: err?.message,
          detail: err?.response?.data
        });
      }
    }

    const isAccepted = passed === testCases.length;

    console.log("[7] Updating problem stats");
    problem.totalSubmissions++;
    if (isAccepted) problem.acceptance++;
    problem.acceptanceRate = (problem.acceptance / problem.totalSubmissions) * 100;
    await problem.save();

    console.log("[8] Fetching user:", req.user._id);
    const user = await User.findById(req.user._id);
    if (!user) return res.status(404).json({ message: "User not found" });

    console.log("[9] Updating user stats");
    user.totalSubmissions++;
    if (isAccepted) {
      user.acceptedSubmissions++;
      if (
```

```
107:         problem.problemNumber != null &&
108:         Array.isArray(user.problemsSolved) &&
109:         !user.problemsSolved.includes(problem.problemNumber)
110:       ) {
111:         user.problemsSolved.push(problem.problemNumber);
112:       }
113:     }
114:     await user.save();
115:
116:     console.log("[10] Creating submission");
117:     const submissionData = {
118:       userId: user._id,
119:       problemId: problem._id,
120:       language,
121:       code,
122:       input: testCases.map((tc) => tc.input).join("\n---\n"),
123:       output: testCases.map((tc) => tc.output).join("\n---\n"),
124:       status: isAccepted ? "Accepted" : "Wrong Answer",
125:       executionTime: "N/A",
126:     };
127:
128:     if (problem.problemNumber != null) {
129:       submissionData.problemNumber = problem.problemNumber;
130:     }
131:
132:     const submission = await Submission.create(submissionData);
133:
134:     console.log("[11] Submission saved");
135:     return res.json({ status: submission.status });
136:   } catch (e) {
137:     console.error("[12] Fatal backend error in submitSolution:");
138:     console.dir(e, { depth: null });
139:     return res.status(500).json({
140:       message: "Internal server error",
141:       error: e.message,
142:       stack: e.stack,
143:     });
144:   }
145: };
```

Walkthrough

Lines 9-10: Defines output normalization for comparison.

Lines 11-13: Starts all-submissions history route.

Lines 14-21: Loads current user's submission history.

Lines 22-13: Starts per-problem history route.

Lines 14-39: Filters history by user and problem.

Line 40: Starts the main judge workflow.

Lines 41-44: Rejects invalid input before expensive work.

Lines 45-48: Reads the target problem id.

Lines 49-52: Extracts validated code and language.

Lines 53-56: Loads the target problem.

Lines 57-61: Loads hidden judge test cases.

Lines 62-64: Determines compiler service base URL.

Lines 65-67: Executes the submitted code against each test case.

Lines 68-74: Calls compiler service `/run`.

Lines 75-89: Normalizes actual and expected output before comparison.

Lines 90-92: Computes final verdict.

Lines 93-98: Updates problem metrics.

Lines 99-102: Loads user to update personal stats.

Lines 103-116: Updates user counters and solved problems.

Lines 117-131: Builds submission document data.

Lines 132-134: Persists the submission record.

Lines 135-145: Returns final verdict to client.

Test Case Controllers

File: Backend/src/Controllers/testCaseControllers.mjs

Purpose: Implements admin test-case creation and fetch logic.

How it is used in request flow: Runs after auth/admin middleware and validation, then talks directly to the `TestCase` model.

What to say in interview:

- Uses bulk insert with `insertMany`.
- Maps route `:id` into each new test-case document.

Implementation notes:

- Bulk insert is efficient, but there is no transaction around multi-document changes.

Explain it simply in interview: This controller file contains the main business logic for test case. Routes send validated requests here, and controllers talk to models, utilities, and the response object.

Likely Akamai follow-up questions:

- What belongs in a controller and what should stay in middleware?
- Why validate input before controller logic starts?
- What improvement would you mention first in this controller?

One line to remember: Controllers are where request data turns into actual application behavior.

Code

```
import { matchedData, validationResult } from "express-validator";
import mongoose from "mongoose";
import { TestCase } from "../Models/Testcase.mjs";

export const addTestCase = async (req, res) => {
  const { id } = req.params;

  if (!mongoose.Types.ObjectId.isValid(id))
    return res.status(400).json({ message: "Invalid problem ID" });

  const errors = validationResult(req);
  if (!errors.isEmpty())
    return res.status(400).json({ message: "Validation failed", errors: errors.array() });

  const { testCases } = matchedData(req, { includeOptionals: true });

  const docs = testCases.map(tc => ({ ...tc, problemId: id }));

  try {
    await TestCase.insertMany(docs);
    res.status(201).json({ message: "Test-cases added" });
  } catch (err) {
    res.status(500).json({ message: "Insert failed", error: err.message });
  }
};

export const getTestCasesByProblem = async (req, res) => {
  const { id } = req.params;

  if (!mongoose.Types.ObjectId.isValid(id))
    return res.status(400).json({ message: "Invalid problem ID" });

  try {
    const testCases = await TestCase.find({ problemId: id });
    res.json({ testCases });
  } catch (err) {
    res.status(500).json({ message: "Fetch failed", error: err.message });
  }
};
```

Walkthrough

Lines 5-7: Starts bulk insert flow.

Lines 8-10: Rejects malformed problem ids.

Lines 11-14: Stops invalid input.

Lines 15-16: Extracts validated test-case payload.

Lines 17-19: Attaches parent problem id to each test case.

Lines 20-26: Bulk inserts test cases.

Lines 27-33: Starts fetch-by-problem flow.

Lines 34-39: Reads all test cases for a problem.

Register Validation Schema

File: Backend/src/Utils/Validating Schemas/Auth/registerValidatingSchema.mjs

Purpose: Validates registration input before controller logic runs.

How it is used in request flow: Rejects bad or duplicate registration data at route level.

What to say in interview:

- Uses DB-backed custom validators for unique username/email.
- Protects admin registration with `ADMIN_SECRET`.

Implementation notes:

- Role is optional and defaults later in controller.

Explain it simply in interview: This schema rejects bad input early, so controllers receive cleaner and safer data. It is the first input safety net before business logic runs.

Likely Akamai follow-up questions:

- Why validate at route level before the controller?
- What is the benefit of `matchedData` after validation?
- Which checks should also exist at the database level?

One line to remember: Validation is the first defense against malformed or malicious input.

Code

```
import { User } from "../../../Models/User.mjs";

export const registerValidatingSchema = {
    userName: {
        notEmpty: {
            errorMessage: "User Name cannot be empty",
        },
        isString: {
            errorMessage: "User Name should be a string",
        },
        custom: {
            options: async (value) => {
                const exists = await User.findOne({ userName: value });
                if (exists) {
                    throw new Error("User Name already in use.");
                }
                return true;
            },
        },
    },

    email: {
        notEmpty: {
            errorMessage: "Email cannot be empty",
        },
        isEmail: {
            errorMessage: "Invalid Email",
        },
        custom: {
            options: async (value) => {
                const exists = await User.findOne({ email: value });
                if (exists) {
                    throw new Error("Email already registered");
                }
                return true;
            },
        },
    },

    password: {
        notEmpty: {
```

```
42:             errorMessage: "Password cannot be empty",
43:         },
44:         isLength: {
45:             options: {
46:                 min: 8,
47:                 max: 32,
48:             },
49:             errorMessage: "Password must be 8-32 characters long",
50:         },
51:         matches: {
52:             options: [/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).+$/],
53:             errorMessage: "Password must include uppercase, lowercase, number, and special character",
54:         },
55:     },
56:
57:     role: {
58:         optional: true,
59:         isIn: {
60:             options: [['admin', 'user']],
61:             errorMessage: "Role must be 'admin' or 'user'",
62:         },
63:     },
64:
65:     adminSecret: {
66:         optional: true,
67:         custom: {
68:             options: (value, { req }) => {
69:                 const role = req.body?.role?.toLowerCase();
70:
71:                 if (role !== "admin") return true;
72:
73:                 if (!value) {
74:                     throw new Error("Admin secret is required for admin registration");
75:                 }
76:
77:                 if (value !== process.env.ADMIN_SECRET) {
78:                     throw new Error("Invalid admin secret");
79:                 }
80:
81:                 return true;
82:             },
83:         },
84:     },
85: };
```

Walkthrough

Line 3: Defines the registration validation schema.

Lines 4-21: Validates username and uniqueness.

Lines 22-39: Validates email and uniqueness.

Lines 40-56: Validates password length and strength.

Lines 57-64: Validates optional role.

Lines 65-85: Validates admin secret for admin registration.

Login Validation Schema

File: Backend/src/Utils/Validating Schemas/Auth/loginValidatingSchema.mjs

Purpose: Validates login credentials and preloads the matched user.

How it is used in request flow: Runs before login controller and stores the found user on `req._foundUser`.

What to say in interview:

- Validator performs both account lookup and password comparison.
- Optional role check prevents logging in under the wrong declared role.

Implementation notes:

- This pattern keeps the controller small by pushing credential checks into validation.

Explain it simply in interview: This schema rejects bad input early, so controllers receive cleaner and safer data. It is the first input safety net before business logic runs.

Likely Akamai follow-up questions:

- Why validate at route level before the controller?
- What is the benefit of `matchedData` after validation?
- Which checks should also exist at the database level?

One line to remember: Validation is the first defense against malformed or malicious input.

Code

```
import { User } from "../../../Models/User.mjs";
import bcrypt from 'bcryptjs';

export const loginValidatingSchema = {
    email: {
        in: ['body'],
        notEmpty: {
            errorMessage: "Email cannot be empty",
        },
        isEmail: {
            errorMessage: "Invalid Email",
        },
        custom: {
            options: async (value, { req }) => {
                const user = await User.findOne({ email: value });
                if (!user) {
                    throw new Error("Email not registered");
                }

                req._foundUser = user;
                return true;
            },
        },
    },

    password: {
        in: ['body'],
        notEmpty: {
            errorMessage: "Password cannot be empty",
        },
        custom: {
            options: async (value , {req}) => {
                const user = req._foundUser;
                if (!user) return true;

                const isMatch = await bcrypt.compare(value, user.password);
                if (!isMatch) {
                    throw new Error("Incorrect password");
                }

                return true;
```

```
42:             },
43:         },
44:     },
45:
46:     role: {
47:         in: ['body'],
48:         optional: true,
49:         isIn: {
50:             options: [['admin', 'user']],
51:             errorMessage: "Role must be 'admin' or 'user'",
52:         },
53:         custom: {
54:             options: (value, { req }) => {
55:                 const user = req._foundUser;
56:                 if (!value || !user) return true;
57:
58:                 if (user.role?.toLowerCase() !== value.toLowerCase()) {
59:                     throw new Error(`This account is not registered as ${value}`);
60:                 }
61:
62:                 return true;
63:             },
64:         },
65:     },
66: };
67:
68: export default loginValidatingSchema
```

Walkthrough

Line 4: Defines the login validation schema.

Lines 5-25: Validates email and loads the matching user.

Lines 26-45: Compares submitted password with stored hash.

Lines 46-67: Validates optional role against stored account role.

Line 68: Exports the schema.

Problem Validation Schema

File: Backend/src/Utils/Validating Schemas/problemValidatingSchema.mjs

Purpose: Validates problem-creation payloads.

How it is used in request flow: Ensures only well-formed problem data reaches `addProblem` controller.

What to say in interview:

- Uses nested validation for examples array.
- Checks uniqueness of both problem number and title.

Implementation notes:

- Converts `problemNumber` to int before controller use.

Explain it simply in interview: This schema rejects bad input early, so controllers receive cleaner and safer data. It is the first input safety net before business logic runs.

Likely Akamai follow-up questions:

- Why validate at route level before the controller?
- What is the benefit of `matchedData` after validation?
- Which checks should also exist at the database level?

One line to remember: Validation is the first defense against malformed or malicious input.

Code

```
import { Problem } from "../../Models/Problem.mjs";

export const problemValidatingSchema = {

    problemNumber: {
        in: ['body'],
        notEmpty: {
            errorMessage: "Problem number is required",
        },
        isInt: {
            errorMessage: "Problem number must be an integer",
        },
        custom: {
            options: async (value) => {
                const exists = await Problem.findOne({ problemNumber: value });
                if (exists) {
                    throw new Error("Problem number already exists");
                }
                return true;
            },
        },
        toInt: true,
    },

    title: {
        in: ['body'],
        notEmpty: {
            errorMessage: "Title is required",
        },
        isString: {
            errorMessage: "Title must be a string",
        },
        custom: {
            options: async (value) => {
                const exists = await Problem.findOne({ title: value });

                if (exists) {
                    throw new Error("Problem title already exists");
                }

                return true;
```

```
42:             },
43:         },
44:     },
45:
46:     statement: {
47:         in: ['body'],
48:         notEmpty: {
49:             errorMessage: "Statement is required",
50:         },
51:         isString: {
52:             errorMessage: "Statement must be a string",
53:         },
54:     },
55:
56:     examples: {
57:         in: ['body'],
58:         isArray: {
59:             errorMessage: "Examples must be an array",
60:         },
61:     },
62:
63:     "examples.*.input": {
64:         in: ['body'],
65:         isString: {
66:             errorMessage: "Each example input must be a string",
67:         },
68:     },
69:
70:     "examples.*.output": {
71:         in: ['body'],
72:         isString: {
73:             errorMessage: "Each example output must be a string",
74:         },
75:     },
76:
77:     "examples.*.explanation": {
78:         in: ['body'],
79:         optional: true,
80:         isString: {
81:             errorMessage: "Each explanation must be a string",
82:         },
83:     },
84:
85:     constraints: {
86:         in: ['body'],
87:         notEmpty: {
88:             errorMessage: "Constraints are required",
89:         },
90:         isString: {
91:             errorMessage: "Constraints must be a string",
92:         },
93:     },
94: };
```

Walkthrough

Lines 3-4: Defines the problem validation schema.

Lines 5-24: Validates integer problem number and uniqueness.

Lines 25-45: Validates title and uniqueness.

Lines 46-55: Validates problem statement.

Lines 56-62: Validates examples array.

Lines 63-69: Validates each example input.

Lines 70-84: Validates each example output.

Lines 85-94: Validates constraints.

Profile Validation Schema

File: Backend/src/Utils/Validating Schemas/profileValidatingSchema.mjs

Purpose: Validates incoming profile updates.

How it is used in request flow: Runs before profile update controller.

What to say in interview:

- All fields are optional to support partial updates.
- Profile picture is validated as a URL.

Implementation notes:

- `firstName` and `lastName` are validated but never stored.

Explain it simply in interview: This schema rejects bad input early, so controllers receive cleaner and safer data. It is the first input safety net before business logic runs.

Likely Akamai follow-up questions:

- Why validate at route level before the controller?
- What is the benefit of `matchedData` after validation?
- Which checks should also exist at the database level?

One line to remember: Validation is the first defense against malformed or malicious input.

Code

```
export const profileValidatingSchema = {
  firstName: {
    optional: true,
    isString: {
      errorMessage: "First name must be a string"
    },
    trim: true
  },
  lastName: {
    optional: true,
    isString: {
      errorMessage: "Last name must be a string"
    },
    trim: true
  },
  profilePicture: {
    optional: true,
    isURL: {
      errorMessage: "Profile picture must be a valid URL"
    }
  },
  bio: {
    optional: true,
    isString: {
      errorMessage: "Bio must be a string"
    }
  },
  location: {
    optional: true,
    isString: {
      errorMessage: "Location must be a string"
    }
  },
  institution: {
    optional: true,
    isString: {
      errorMessage: "Institution must be a string"
    }
  }
};
```

Walkthrough

Line 1: Defines the profile validation schema.

Lines 2-8: Validates optional first name.

Lines 9-15: Validates optional last name.

Lines 16-21: Validates profile image URL.

Lines 22-27: Validates bio.

Lines 28-33: Validates location.

Lines 34-40: Validates institution.

Submission Validation Schema

File: Backend/src/Utils/Validating Schemas/submissionValidatingSchema.mjs

Purpose: Validates code and language for code-submission requests.

How it is used in request flow: Runs before expensive judge logic starts.

What to say in interview:

- Early validation matters because submit flow is costly.
- Problem id comes from URL, so body only needs code and language.

Implementation notes:

- Language is not restricted to a fixed allow-list here.

Explain it simply in interview: This schema rejects bad input early, so controllers receive cleaner and safer data. It is the first input safety net before business logic runs.

Likely Akamai follow-up questions:

- Why validate at route level before the controller?
- What is the benefit of `matchedData` after validation?
- Which checks should also exist at the database level?

One line to remember: Validation is the first defense against malformed or malicious input.

Code

```
 1: export const submissionValidatingSchema = {
 2:
 3:   code: {
 4:     in: ["body"],
 5:     isString: {
 6:       errorMessage: "Code must be a string"
 7:     },
 8:     notEmpty: {
 9:       errorMessage: "Code is required"
10:     }
11:   },
12:
13:   language: {
14:     in: ["body"],
15:     isString: {
16:       errorMessage: "Language must be a string"
17:     },
18:     notEmpty: {
19:       errorMessage: "Language is required"
20:     }
21:   }
22:
23: };
```

Walkthrough

Lines 1-2: Defines submission validation.

Lines 3-12: Validates code presence and type.

Lines 13-23: Validates language presence and type.

Test Case Validation Schema

File: Backend/src/Utils/Validating Schemas/testCaseValidatingSchema.mjs

Purpose: Validates admin test-case payloads before bulk insert.

How it is used in request flow: Runs before `addTestCase` controller.

What to say in interview:

- Validates both top-level array and nested test-case fields.
- Keeps controller logic simple by guaranteeing payload shape.

Explain it simply in interview: This schema rejects bad input early, so controllers receive cleaner and safer data. It is the first input safety net before business logic runs.

Likely Akamai follow-up questions:

- Why validate at route level before the controller?
- What is the benefit of `matchedData` after validation?
- Which checks should also exist at the database level?

One line to remember: Validation is the first defense against malformed or malicious input.

Code

```
 1: export const testCaseValidatingSchema = {
 2:
 3:   "testCases": {
 4:     in: ["body"],
 5:     isArray: {
 6:       errorMessage: "testCases must be an array"
 7:     },
 8:     notEmpty: {
 9:       errorMessage: "Provide at least one test-case"
10:     }
11:   },
12:
13:   "testCases.*.input": {
14:     isString: {
15:       errorMessage: "Each input must be a string"
16:     },
17:     notEmpty: {
18:       errorMessage: "Input is required"
19:     }
20:   },
21:
22:   "testCases.*.output": {
23:     isString: {
24:       errorMessage: "Each output must be a string"
25:     },
26:     notEmpty: {
27:       errorMessage: "Output is required"
28:     }
29:   }
30:
31: };
```

Walkthrough

Lines 1-2: Defines test-case validation.

Lines 3-12: Validates the array container.

Lines 13-21: Validates each input field.

Lines 22-31: Validates each output field.

User Model

File: Backend/src/Models/User.mjs

Purpose: Stores login identity, role, profile information, and progress metrics.

How it is used in request flow: Used by auth, profile, and submission logic across the whole backend.

What to say in interview:

- Combines auth fields and gamification/profile fields in one document.
- Unique `userName` and `email` enforce account uniqueness.

Implementation notes:

- The schema has `userName`, not `name`, which matters because JWT helper references `user.name`.

Explain it simply in interview: This Mongoose model defines the shape of one MongoDB collection. Controllers use it to read, write, and enforce schema-level rules on persistent data.

Likely Akamai follow-up questions:

- Why use a schema instead of free-form JSON documents?
- Which fields here deserve indexes or uniqueness constraints?
- What data would you normalize or split out later if the app grows?

One line to remember: Models define the shape and integrity rules of stored data.

Code

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  userName: {
    type: String,
    required: true,
    unique: true,
    default: null,
  },
  email: {
    type: String,
    required: true,
    unique: true,
    default: null,
  },
  password: {
    type: String,
    required: true,
  },
  role: {
    type: String,
    enum: ['user', 'admin'],
    required: true,
  },
  profilePicture: {
    type: String,
    default: null
  },
  bio: {
    type: String,
    default: null
  },
  location: {
    type: String,
    default: null
  },
  institution: {
    type: String,
    default: null
  },

```

```
42:   problemsSolved: {
43:     type: [Number],
44:     default: []
45:   },
46:   totalSubmissions: {
47:     type: Number,
48:     default: 0
49:   },
50:   acceptedSubmissions: {
51:     type: Number,
52:     default: 0
53:   }
54: });
55:
56: userSchema.pre('save', function (next) {
57:   if (this.role) {
58:     this.role = this.role.toLowerCase();
59:   }
60:   next();
61: });
62:
63: export const User = mongoose.models.User || mongoose.model("User", userSchema);
```

Walkthrough

Line 3: Starts the user schema definition.

Lines 4-9: Stores the public username/handle.

Lines 10-15: Stores the login email with uniqueness.

Lines 16-19: Stores hashed password data.

Lines 20-24: Stores `user` or `admin`.

Lines 25-41: Begins optional profile fields.

Lines 42-55: Stores solved problem numbers and aggregate stats.

Lines 56-62: Normalizes role to lowercase before save.

Line 63: Exports the reusable model.

Problem Model

File: Backend/src/Models/Problem.mjs

Purpose: Stores coding problem content, examples, constraints, creator, and acceptance stats.

How it is used in request flow: Used by problem controllers and updated again by the submission controller.

What to say in interview:

- Examples are embedded subdocuments because they belong tightly to a problem.
- Acceptance rate is a derived field stored for read efficiency.

Implementation notes:

- Deleting a problem requires manual cleanup of test cases and submissions.

Explain it simply in interview: This Mongoose model defines the shape of one MongoDB collection. Controllers use it to read, write, and enforce schema-level rules on persistent data.

Likely Akamai follow-up questions:

- Why use a schema instead of free-form JSON documents?
- Which fields here deserve indexes or uniqueness constraints?
- What data would you normalize or split out later if the app grows?

One line to remember: Models define the shape and integrity rules of stored data.

Code

```
 1: import mongoose from "mongoose";
 2:
 3: const exampleSchema = new mongoose.Schema({
 4:     input: {
 5:         type: String,
 6:         required: true,
 7:     },
 8:     output: {
 9:         type: String,
10:         required: true,
11:     },
12:     explanation: {
13:         type: String,
14:         required: false,
15:         default: ""
16:     }
17: });
18:
19: const problemSchema = new mongoose.Schema({
20:     problemNumber: {
21:         type: Number,
22:         required: true,
23:         unique: true,
24:     },
25:     title: {
26:         type: String,
27:         required: true,
28:         unique: true,
29:     },
30:     statement: {
31:         type: String,
32:         required: true,
33:     },
34:     examples: {
35:         type: [exampleSchema],
36:         default: [],
37:     },
38:     constraints: {
39:         type: String,
40:         required: true,
41:     },
```

```
42:     totalSubmissions: {
43:         type: Number,
44:         default: 0,
45:     },
46:     acceptance: {
47:         type: Number,
48:         default: 0,
49:     },
50:     acceptanceRate: {
51:         type: Number,
52:         default: 0,
53:     },
54:     created_by: {
55:         type: mongoose.Schema.Types.ObjectId,
56:         required: true,
57:         ref: 'User',
58:     },
59: });
60:
61: export const Problem = mongoose.models.Problem || mongoose.model("Problem", problemSchema);
```

Walkthrough

Lines 3-18: Defines the embedded example structure.

Line 19: Defines the problem document.

Lines 20-24: Stores a unique numeric identifier.

Lines 25-29: Stores a unique problem title.

Lines 30-33: Stores the problem statement.

Lines 34-37: Stores example input/output pairs.

Lines 38-41: Stores textual constraints.

Lines 42-53: Begins stored statistics fields.

Lines 54-60: Links the problem to the admin who created it.

Line 61: Exports the Problem model.

Test Case Model

File: Backend/src/Models/Testcase.mjs

Purpose: Stores hidden input/output data used by the judge.

How it is used in request flow: Loaded by submission controller and managed through admin-only test-case routes.

What to say in interview:

- Hidden judge data is separated from public problem content.
- Each test case points to its parent problem through `problemId`.

Implementation notes:

- An index on `problemId` would help repeated lookup during judging.

Explain it simply in interview: This Mongoose model defines the shape of one MongoDB collection. Controllers use it to read, write, and enforce schema-level rules on persistent data.

Likely Akamai follow-up questions:

- Why use a schema instead of free-form JSON documents?
- Which fields here deserve indexes or uniqueness constraints?
- What data would you normalize or split out later if the app grows?

One line to remember: Models define the shape and integrity rules of stored data.

Code

```
 1: import mongoose from "mongoose";
 2:
 3: const testCaseSchema = new mongoose.Schema({
 4:   problemId: {
 5:     type: mongoose.Schema.Types.ObjectId,
 6:     ref: "Problem",
 7:     required: true
 8:   },
 9:   input: {
10:     type: String, required: true
11:   },
12:   output: {
13:     type: String, required: true
14:   }
15: });
16:
17: export const TestCase = mongoose.models.TestCase || mongoose.model("TestCase", testCaseSchema);
```

Walkthrough

Line 3: Defines the test-case schema.

Lines 4-8: Links the test case to a problem.

Lines 9-11: Stores stdin-style input.

Lines 12-16: Stores expected output.

Line 17: Exports the TestCase model.

Submission Model

File: Backend/src/Models/Submission.mjs

Purpose: Stores each user submission with code, verdict, related ids, and execution metadata.

How it is used in request flow: Written by `submitSolution` and read by submission history endpoints.

What to say in interview:

- Submission documents create an audit trail of every attempt.
- Stored `problemNumber` avoids needing a problem join for simple history views.

Implementation notes:

- `submissionId` based on `Date.now()` is not a strong unique identifier under concurrency.
- Indexes on user/problem lookup paths would help performance.

Explain it simply in interview: This Mongoose model defines the shape of one MongoDB collection. Controllers use it to read, write, and enforce schema-level rules on persistent data.

Likely Akamai follow-up questions:

- Why use a schema instead of free-form JSON documents?
- Which fields here deserve indexes or uniqueness constraints?
- What data would you normalize or split out later if the app grows?

One line to remember: Models define the shape and integrity rules of stored data.

Code

```
import mongoose from "mongoose";

const submissionSchema = new mongoose.Schema({
  submissionId: {
    type: Number,
    default: () => Date.now()
  },
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true
  },
  problemId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Problem",
    required: true
  },
  problemNumber: {
    type: Number,
    required: false
  },
  language: {
    type: String,
    required: true
  },
  code: {
    type: String,
    required: true
  },
  input: {
    type:
      String,
    default: ""
  },
  output:
  {
    type: String
  },
  status: {
    type: String,
```

```
41:     enum: ["Accepted", "Wrong Answer", "Runtime Error", "Compilation Error", "Pending"],
42:     default: "Pending"
43:   },
44:   executionTime: {
45:     type: String
46:   },
47:   createdAt: {
48:     type: Date,
49:     default: Date.now
50:   }
51: });
52:
53: export const Submission = mongoose.models.Submission || mongoose.model("Submission", submissionSchema);
```

Walkthrough

Line 3: Defines the submission document.

Lines 4-7: Adds an app-level timestamp-based id.

Lines 8-12: Links submission to user.

Lines 13-17: Links submission to problem.

Lines 18-21: Stores a copied problem number.

Lines 22-25: Stores submission language.

Lines 26-29: Stores submitted source code.

Lines 30-38: Stores judge input snapshot.

Lines 39-46: Stores result verdict.

Lines 47-52: Stores creation timestamp.

Line 53: Exports the model.

Auth Routes

File: Backend/src/Routes/authRoutes.mjs

Purpose: Maps auth URLs to auth validation and controller functions.

How it is used in request flow: Incoming auth requests are dispatched here after global middleware completes in `Index.mjs`.

What to say in interview:

- Validation is attached at route level with `checkSchema(...)`.
- Current-user lookup is protected by `requireAuth`.

Implementation notes:

- Router uses full paths rather than a mounted `/api/auth` prefix.

Explain it simply in interview: This route file maps HTTP endpoints to validation, middleware, and controller functions. It is the API traffic map for that feature.

Likely Akamai follow-up questions:

- Why keep routes thin and move logic into controllers?
- Where do validation and authentication fit in the route chain?
- Would you keep full paths here or mount a common prefix? why?

One line to remember: Routes are small, but they define the public API contract of the backend.

Code

```
 1: import { Router } from 'express';
 2: import { login, logout, register, me } from '../Controllers/authControllers.mjs';
 3: import { registerValidatingSchema } from '../Utils/Validating Schemas/Auth/registerValidatingSchema.mjs';
 4: import { checkSchema } from "express-validator";
 5: import loginValidatingSchema from '../Utils/Validating Schemas/Auth/loginValidatingSchema.mjs';
 6: import requireAuth from '../Middleware/requireAuth.mjs';
 7:
 8: const router = Router();
 9:
10: router.post('/api/auth/register' , checkSchema(registerValidatingSchema) , register);
11:
12: router.post('/api/auth/login' , checkSchema(loginValidatingSchema) , login);
13:
14: router.get('/api/auth/me', requireAuth, me);
15:
16: router.post('/api/auth/logout' , logout);
17:
18: export default router;
```

Walkthrough

Lines 8-9: Creates the router instance.

Lines 10-11: Register route with validation.

Lines 12-13: Login route with validation.

Lines 14-15: Authenticated current-user route.

Lines 16-17: Logout route.

Line 18: Exports the router.

Problem Routes

File: Backend/src/Routes/problemRoutes.mjs

Purpose: Maps problem listing, read, create, and delete endpoints.

How it is used in request flow: Problem routes call public controllers directly or run auth/admin/validation middleware first.

What to say in interview:

- Middleware order is `requireAuth` then `isAdmin` for sensitive routes.
- Route design mixes plural collection path and singular resource path.

Implementation notes:

- Single problem read is protected, while problem list is public.

Explain it simply in interview: This route file maps HTTP endpoints to validation, middleware, and controller functions. It is the API traffic map for that feature.

Likely Akamai follow-up questions:

- Why keep routes thin and move logic into controllers?
- Where do validation and authentication fit in the route chain?
- Would you keep full paths here or mount a common prefix? why?

One line to remember: Routes are small, but they define the public API contract of the backend.

Code

```
 1: import { Router } from "express";
 2: import { addProblem, deleteProblem, getProblem, getProblemSet } from
"../Controllers/problemControllers.mjs";
 3: import requireAuth from "../Middleware/requireAuth.mjs";
 4: import isAdmin from "../Middleware/isAdmin.mjs";
 5: import { checkSchema } from "express-validator";
 6: import { problemValidatingSchema } from "../Utils/Validating Schemas/problemValidatingSchema.mjs";
 7:
 8: const router = Router();
 9:
10: router.get('/api/problems' , getProblemSet);
11:
12: router.get('/api/problem/:Id' , requireAuth , getProblem);
13:
14: router.post('/api/problem' , requireAuth , isAdmin , checkSchema(problemValidatingSchema) , addProblem);
15:
16: router.delete('/api/problem/:Id' , requireAuth , isAdmin , deleteProblem);
17:
18: export default router;
```

Walkthrough

Lines 8-9: Creates the problem router.

Lines 10-11: Public list route.

Lines 12-13: Authenticated single-problem route.

Lines 14-15: Admin-only create route.

Lines 16-17: Admin-only delete route.

Line 18: Exports the router.

Test Case Routes

File: Backend/src/Routes/testCaseRoutes.mjs

Purpose: Maps admin-only test-case creation and read endpoints.

How it is used in request flow: Every request on this router passes auth and admin authorization before touching hidden judge data.

What to say in interview:

- Keeping hidden test cases admin-only protects judge integrity.
- Bulk insert route validates the request payload before controller execution.

Implementation notes:

- Uses `:id` param casing unlike problem routes' `:Id`.

Explain it simply in interview: This route file maps HTTP endpoints to validation, middleware, and controller functions. It is the API traffic map for that feature.

Likely Akamai follow-up questions:

- Why keep routes thin and move logic into controllers?
- Where do validation and authentication fit in the route chain?
- Would you keep full paths here or mount a common prefix? why?

One line to remember: Routes are small, but they define the public API contract of the backend.

Code

```
 1: import { Router } from "express";
 2: import { checkSchema } from "express-validator";
 3: import { testCaseValidatingSchema } from "../Utils/Validating Schemas/testCaseValidatingSchema.mjs";
 4: import { addTestCase, getTestCasesByProblem } from "../Controllers/testCaseControllers.mjs";
 5: import requireAuth from "../Middleware/requireAuth.mjs";
 6: import isAdmin from "../Middleware/isAdmin.mjs";
 7:
 8: const router = Router();
 9:
10: router.post("/api/testcases/:id", requireAuth, isAdmin, checkSchema(testCaseValidatingSchema),
addTestCase);
11:
12: router.get("/api/testcases/:id", requireAuth, isAdmin, getTestCasesByProblem);
13:
14: export default router;
```

Walkthrough

Lines 8-9: Creates the test-case router.

Lines 10-11: Admin-only create route.

Lines 12-13: Admin-only read route.

Line 14: Exports the router.

Profile Routes

File: Backend/src/Routes/profileRoutes.mjs

Purpose: Maps leaderboard, public profile, self-profile, and profile update endpoints.

How it is used in request flow: Public profile routes bypass auth while self-profile routes require it.

What to say in interview:

- Leaderboard is intentionally public.
- Profile update uses validation before controller logic.

Implementation notes:

- `/api/profile` is the current-user resource; `/api/user/:id` is a public user resource.

Explain it simply in interview: This route file maps HTTP endpoints to validation, middleware, and controller functions. It is the API traffic map for that feature.

Likely Akamai follow-up questions:

- Why keep routes thin and move logic into controllers?
- Where do validation and authentication fit in the route chain?
- Would you keep full paths here or mount a common prefix? why?

One line to remember: Routes are small, but they define the public API contract of the backend.

Code

```
 1: import { Router } from "express";
 2: import requireAuth from "../Middleware/requireAuth.mjs";
 3: import { checkSchema } from "express-validator";
 4: import { profileValidatingSchema } from "../Utils/Validating Schemas/profileValidatingSchema.mjs";
 5: import {
 6:   editProfile,
 7:   getLeaderboard,
 8:   getMyProfile,
 9:   userProfile
10: } from "../Controllers/profileControllers.mjs";
11:
12: const router = Router();
13:
14: router.get("/api/leaderboard", getLeaderboard);
15:
16: router.get("/api/user/:id" , userProfile);
17:
18: router.get("/api/profile" , requireAuth  , getMyProfile);
19:
20: router.put("/api/profile" , requireAuth, checkSchema(profileValidatingSchema), editProfile);
21:
22: export default router;
```

Walkthrough

Lines 12-13: Creates the profile router.

Lines 14-15: Public leaderboard route.

Lines 16-17: Public user profile route.

Lines 18-19: Authenticated own-profile route.

Lines 20-21: Authenticated profile update route.

Line 22: Exports the router.

Submission Routes

File: Backend/src/Routes/submissionRoutes.mjs

Purpose: Maps submission history and code-judging endpoints.

How it is used in request flow: All routes require auth; the submit route also validates request body and triggers the deepest controller flow in the backend.

What to say in interview:

- These routes are performance-sensitive because submit path touches DB and external compiler service.
- History endpoints scope results to the logged-in user.

Implementation notes:

- Submit route is where most interview discussion about request flow should focus.

Explain it simply in interview: This route file maps HTTP endpoints to validation, middleware, and controller functions. It is the API traffic map for that feature.

Likely Akamai follow-up questions:

- Why keep routes thin and move logic into controllers?
- Where do validation and authentication fit in the route chain?
- Would you keep full paths here or mount a common prefix? why?

One line to remember: Routes are small, but they define the public API contract of the backend.

Code

```
 1: import { Router } from "express";
 2: import { checkSchema } from "express-validator";
 3: import { submissionValidatingSchema } from "../Utils/Validating Schemas/submissionValidatingSchema.mjs";
 4: import { showAllSubmissions, showProblemSubmissions, submitSolution } from
"../Controllers/submissionControllers.mjs";
 5: import requireAuth from "../Middleware/requireAuth.mjs";
 6:
 7: const router = Router();
 8:
 9: router.get("/api/submissions", requireAuth, showAllSubmissions);
10:
11: router.get("/api/problem/:id/submissions", requireAuth, showProblemSubmissions);
12:
13: router.post("/api/problem/:id/submit", requireAuth, checkSchema(submissionValidatingSchema),
submitSolution);
14:
15: export default router;
```

Walkthrough

Lines 7-8: Creates the submission router.

Lines 9-10: All-submissions history route.

Lines 11-12: Per-problem history route.

Lines 13-14: Judging route.

Line 15: Exports the router.

Compiler Service Appendix

The backend submission flow depends on a separate compiler service. Understanding the contract helps you explain how code travels beyond the main API.

Explain it simply in interview: This section matters because the backend does not execute user code by itself. It depends on a separate compiler service, so you should clearly explain the contract between the two services.

Likely Akamai follow-up questions:

- Why separate code execution from the main backend?
- What security risks exist when executing user-submitted code?
- What request/response contract does the backend expect from `/run`?

One line to remember: The backend decides the verdict, but the compiler service actually executes the submitted code.

Compiler Service Entrypoint

File: Compiler/Index.js

Purpose: Receives code-execution requests from backend and dispatches execution by language.

Backend dependency: Backend submission controller calls `POST /run` here for every hidden test case.

```
 1: const express = require('express');
 2: const app = express();
 3: const { generateFile } = require('./generateFile');
 4: const { generateInputFile } = require('./generateInputFile');
 5: const cors = require('cors');
 6: const { aiCodeReview } = require('./aiCodeReview.js');
 7: const { executeCpp } = require('./executeCpp');
 8: const { executePython } = require('./executePython');
 9: const { executeJs } = require('./executeJs');
10: const { executeGo } = require('./executeGo');
11: const { executeC } = require('./executeC.js');
12:
13: app.use(cors());
14: app.use(express.urlencoded({ extended: true }));
15: app.use(express.json());
16:
17: app.get("/", (req, res) => {
18:     res.json({ online: 'compiler' });
19: });
20:
21: app.post("/run", async (req, res) => {
22:     const { language = 'cpp', code, input = '' } = req.body;
23:
24:     if (code === undefined || code.trim() === '') {
25:         return res.status(400).json({
26:             success: false,
27:             error: "Empty code! Please provide some code to execute."
28:         });
29:     }
30:
31:     try {
32:         const filePath = await generateFile(language, code);
33:
34:         const inputPath = await generateInputFile(input);
35:
36:         let output;
37:
38:         if (language === "cpp") {
39:             output = await executeCpp(filePath, inputPath);
40:         } else if (language === "c") {
41:             output = await executeC(filePath, inputPath);
42:         } else if (language === "python") {
43:             output = await executePython(filePath, inputPath);
```

```
44:         } else if (language === "javascript") {
45:             output = await executeJs(filePath, inputPath);
46:         } else if (language === "go") {
47:             output = await executeGo(filePath, inputPath);
48:         } else {
49:             throw new Error("Unsupported language");
50:         }
51:
52:         res.json({
53:             success: true,
54:             filePath,
55:             inputPath,
56:             output
57:         });
58:     } catch (error) {
59:         console.error('Error executing code:', error);
60:
61:         res.status(500).json({
62:             success: false,
63:             error: error.message || error.toString() || 'An error occurred while executing the code'
64:         });
65:     }
66: });
67:
68: app.post("/ai-review", async (req, res) => {
69:     const { code } = req.body;
70:     console.log("Code received for review:", code);
71:
72:     if (!code) {
73:         return res.status(400).json({ error: "Code is required" });
74:     }
75:
76:     try {
77:         const review = await aiCodeReview(code);
78:         console.log("AI Review output:", review);
79:         res.json({ review });
80:     } catch (error) {
81:         console.error("AI Review error:", error);
82:         res.status(500).json({ error: "AI Review failed: " + error.message });
83:     }
84: });
85:
86: const PORT = process.env.PORT || 7000;
87: app.listen(PORT, () => {
88:     console.log(`Server is listening on port ${PORT}!`);
89: });
```

Lines 1-16: Bootstraps the compiler service and loads helpers.

Lines 17-20: Defines a health endpoint.

Lines 21-31: Defines the execution endpoint used by the backend.

Lines 32-33: Writes source code to disk.

Lines 34-37: Writes stdin data to disk.

Lines 38-17: Branches to the appropriate runtime handler.

Lines 18-67: Returns output to the backend.

Lines 68-89: Defines an AI review endpoint separate from judge execution.

Compiler Source File Generator

File: Compiler/generateFile.js

Purpose: Creates temporary source files for the compiler service.

Backend dependency: Called by compiler service before code execution.

```
1: const fs = require('fs');
2: const path = require('path');
3: const { v4: uuid } = require('uuid');
4:
```

```
 5: const dirCodes = path.join(__dirname, 'codes');
 6:
 7: if (!fs.existsSync(dirCodes)) {
 8:     fs.mkdirSync(dirCodes, { recursive: true });
 9: }
10:
11: const generateFile = (format, content) => {
12:   const jobID = uuid();
13:   const filename = format === "java" ? "Main.java" : `${jobID}.${format}`;
14:   const filePath = path.join(dirCodes, filename);
15:   fs.writeFileSync(filePath, content);
16:   return filePath;
17: };
18:
19: module.exports = {
20:     generateFile,
21: };
```

Lines 1-4: Imports filesystem, path, and uuid helpers.

Lines 5-6: Sets the source-code folder.

Lines 7-10: Creates folder if needed.

Line 11: Defines file-creation helper.

Lines 12-14: Creates a unique job id.

Lines 15-21: Writes code file to disk.

Compiler Input File Generator

File: Compiler/generateInputFile.js

Purpose: Creates temporary input files for program stdin.

Backend dependency: Called by compiler service before execution.

```
 1: const fs = require('fs');
 2: const path = require('path');
 3: const { v4: uuid } = require('uuid');
 4:
 5: const dirInputs = path.join(__dirname, 'inputs');
 6:
 7: if (!fs.existsSync(dirInputs)) {
 8:     fs.mkdirSync(dirInputs, { recursive: true });
 9: }
10:
11: const generateInputFile = async (input) => {
12:     const jobID = uuid();
13:     const input_filename = `${jobID}.txt`;
14:     const input_filePath = path.join(dirInputs, input_filename);
15:     await fs.writeFileSync(input_filePath, input);
16:     return input_filePath;
17: };
18:
19: module.exports = {
20:     generateInputFile,
21: };
```

Lines 1-4: Imports filesystem helpers.

Lines 5-6: Sets the input folder.

Lines 7-10: Creates folder if needed.

Lines 11-14: Defines helper for writing input files.

Lines 15-21: Writes input contents to disk.

Best Practices To Mention In Interviews

- Explain the code as layered flow: entrypoint -> middleware -> routes -> controllers -> models -> database.
- Mention cookie-based JWT auth, not localStorage token auth.
- Mention that hidden judge test cases are separated from visible problem content.
- Mention that submission orchestration is the most complex and performance-sensitive path.
- Mention improvements: transactions, indexes, global error middleware, rate limiting, logging, and tests.

Common Bugs And How To Talk About Them In Interview

Safe interview strategy: do not hide bugs. Say what the issue is, what impact it can cause, and what fix you would implement first. That sounds mature in interviews.

JWT config bug: `dotenv.config;` is written without parentheses. That means environment variables may not load as intended. Fix: change it to `dotenv.config();` and verify secrets are present at startup.

Wrong JWT payload field: Token creation uses `user.name`, but the schema stores `userName`. This can put incorrect identity data inside the token. Fix: use the exact schema field consistently.

No centralized error middleware: Many routes handle errors locally or inconsistently. Fix: add one global Express error middleware plus better logging.

Submission updates are not transactional: Problem stats, user stats, and submission history are updated separately. A mid-request failure can leave data inconsistent. Fix: use MongoDB transactions where possible.

History endpoints lack `try/catch` in places: Unexpected failures can leak as unhandled errors. Fix: wrap DB calls properly and return consistent API error responses.

Missing performance indexes: Collections like submissions and test cases can become slow as data grows. Fix: add indexes for the main query paths used by controllers.

Cookie/signing inconsistency: Cookie parser gets a secret, but the auth cookie is not used as a signed cookie. Fix: either sign the cookie or remove the misleading configuration for clarity.

If interviewer asks “what would you improve first?”: Say: first I would fix auth correctness bugs, then add global error handling, then make submission/stat updates transactional because that path is the most business-critical.